

LLM Observability & Evaluation Platform

A lightweight, self-hosted LLM observability platform for engineering teams: log every LLM call as a trace, inspect it step by step, and judge whether the answer was any good — all from one shared dashboard, without needing a hosted third-party platform or a large stack.

1. What We Solved

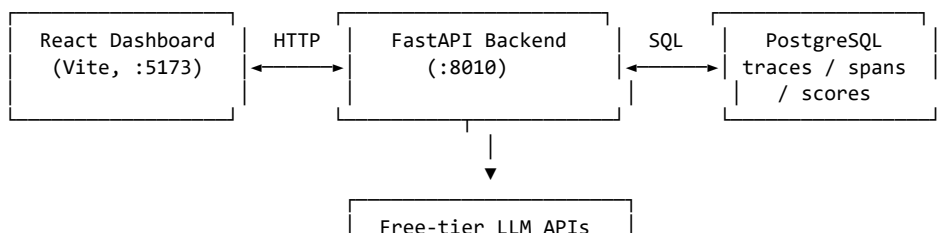
When a team builds anything on top of an LLM — a chatbot, a RAG pipeline, an agent — they immediately lose shared visibility into what's actually happening:

- **No one can see what the model was actually asked or actually answered.** Once a request is done, it's gone unless it's logged centrally, somewhere the whole team can check.
- **The team doesn't know what it costs.** Token usage and dollar cost per request are invisible by default, which makes it hard to catch a runaway bill before it happens.
- **Multi-step pipelines are a black box.** If a RAG pipeline gives a bad answer, was it the retrieval step, the embedding step, or the generation step that failed? Without breaking a request into steps, nobody can tell — including whoever gets paged about it.
- **Errors are cryptic.** A raw API error ("429 rate limit", a malformed JSON parse failure) doesn't tell whoever's on call *what actually went wrong* or *what to do about it*, so debugging knowledge doesn't transfer across the team.
- **The team can't tell if a prompt or model change made things better or worse.** Without a repeatable way to test a set of questions against a model and score the answers, every change is a guess — and there's no shared record of what was tried.

This project addresses all five: every request is logged as a **trace**, every step inside it as a **span**, every dollar and token is recorded, errors are automatically translated into plain language, and there's a built-in evaluation suite that scores answers with an LLM judge — not just one person eyeballing outputs. All of it lands in one shared dashboard, so the whole team is looking at the same data.

2. How It Works

Architecture:



Data flow, end to end:

1. A request happens — through the **Playground** (a real chat UI), the **Evaluation** page (a batch test runner), or your own code calling the logging SDK (`log_trace.py`). 2. The backend creates one row in `traces` (`POST /traces`): what was asked, what came back, when it started/ended, tokens used, and cost. 3. If the request has multiple internal steps (embed → retrieve → generate), each is logged as its own row in `spans` (`POST /spans`), linked to the parent trace by `trace_id`. 4. If a span fails, its raw `error` is sent to an LLM (Groq), which writes back a 2–3 sentence, jargon-free `error_explanation` — automatically, before the response is even returned to the caller. 5. A trace's `status` (`success` / `error` / `pending`) isn't stored — it's computed live from whether any of its spans have an error, and whether the trace has finished. 6. Optionally, an LLM **judge** scores a trace's answer (relevance, faithfulness, hallucination) and the result is saved to `scores`, linked to the trace. 7. The React dashboard reads all of this back through `GET /traces` and `GET /traces/{id}` and renders it as tables, charts, and a full request/response/span inspector.

Every table, endpoint, and UI panel here is something the team wrote and understands — there's no black box, and no per-seat license standing between the team and its own data.

3. Features

Data model (Postgres). Two core tables: `traces` (one row per LLM interaction — name, input, output, timing, token usage, cost) and `spans` (one row per step within a trace — step name, input, output, timing). Spans also carry `error` and `error_explanation`: when a step's `error` is set, the backend automatically asks an LLM to generate a plain-language explanation and a suggested fix before saving the row. A `scores` table (`trace_id`, `score_name`, `score_value`, `explanation`, `created_at`) backs LLM-judged scoring; `score_trace.py` demonstrates the full loop — an LLM judges a real trace's relevance on a 0–1 scale with an explanation, saved to that table.

Backend (FastAPI, `main.py`). Endpoints: `POST/GET /traces`, `GET /traces/{id}` (nested spans, computed status), `POST /spans` (with automatic error explanation), `POST /scores`, `POST /playground/run` (multi-turn chat against a chosen provider), `POST /evaluation/run` / `run_one` (keyword-match and LLM-judge grading), and `GET /providers/status`. `providers.py` wraps three free-tier LLM APIs (Gemini, Groq, OpenRouter) behind one interface, supporting full conversation history and temperature/top-p sampling.

Dashboard (React, `frontend/`):

- **Overview** — total requests, P95 latency, total tokens, and estimated cost tiles, a requests-over-time chart split into success/error series (24H/7D/30D), a provider usage breakdown, and a recent-traces table.
- **Trace Explorer** — a master-detail trace browser: filter by model or status, free-text search, pagination, and a full span waterfall per trace, including any captured error and its auto-generated explanation.

- **Playground** — a real multi-turn chat against any configured provider, with temperature/top-p sliders and a system prompt; every turn is logged as a trace automatically.
- **Evaluation** — a test-case builder that runs each case against one or more providers, grading answers by keyword match and by an LLM judge (faithfulness, relevance, hallucination), with live per-case progress, a model comparison table, and CSV/JSON export.
- **Settings** — light/dark theme toggle and live, `.env`-backed provider-configuration status.

A standalone Streamlit view (`dashboard.py`) also exists as a no-frontend-build alternative for browsing the same trace/span data.

4. What Makes This Different

- **It costs nothing to run.** Every provider integration is free-tier. Commercial tools like Datadog LLM Observability or Langfuse Cloud charge by usage; a team can adopt this without a procurement cycle or a per-seat bill.
 - **Errors are translated, not just logged.** Most observability tools show you the raw stack trace and stop there. This one automatically explains *what probably went wrong and what to try next*, in plain English, at the moment the error happens.
 - **Evaluation is built in, not bolted on.** LLM-as-judge scoring (faithfulness, relevance, hallucination detection) runs in the same tool that captures the trace — no exporting data to a separate eval product.
 - **Every layer is owned.** The data model (`traces/spans/scores`), the API, the provider abstraction, and the dashboard were all written from scratch, table by table and endpoint by endpoint — every piece is something we can explain, extend, or debug, rather than a config file for someone else's platform.
 - **No vendor lock-in.** It's three Postgres tables and a FastAPI app. The data never leaves your own database.
-

Tech Stack

Layer	Technology
Backend	FastAPI, SQLAlchemy, Pydantic
Database	PostgreSQL (via Docker Compose)
LLM Providers	Google Gemini, Groq, OpenRouter (all free-tier)
Frontend	React (Vite), Tailwind CSS, Recharts, React Router
Alternate viewer	Streamlit dashboard (<code>dashboard.py</code>)

5. Near-Term Roadmap

These all extend the current schema and stack — no rebuild required. The project is best understood as a continuous **observe** → **evaluate** → **develop** loop. *Observe* is what's already built (Section 3 above). *Evaluate* is judging what was observed — today that only happens on demand, through hand-built Evaluation test cases or a manual run of `score_trace.py`; the roadmap below closes that gap so real production traces get scored too. *Develop* is acting on what evaluation finds — right now that's entirely manual (read the score, read the explanation, go change the prompt or the code); natural-language querying is the first step toward making that self-service.

- **Scores linked to spans.** Extend `scores` so a score can attach to an individual span, not just a whole trace, and surface scores directly in the Trace Explorer — computed automatically, or triggerable from the UI.
- **Natural-language queries over trace data.** A new endpoint that takes a plain-English question about your own traces/spans/scores ("which provider had the highest error rate this week?"), translates it into a read-only query, and returns both the result and a plain-language explanation.
- **Session-level tracing.** Add a nullable, indexed `session_id` column to `traces` so related traces — every turn of a multi-turn Playground conversation, every step of a longer agent workflow — can be grouped and viewed together.

6. Long-Term Vision (Aspirational)

Not a commitment — just an honest acknowledgment of where tools in this space tend to grow: purpose-built high-performance storage for high-volume trace workloads, fine-grained per-user roles and permissions (today, every team member who can reach the dashboard sees the same shared trace history — there's no login or access tiering yet), and a split between a free core and a paid tier with added scale or support. None of that is built yet. It's named here only so the direction is on record.